

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Regina Silva M

NIM. 2310817220011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Regina Silva Maharatini
NIM : 2310817220011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL.....	6
A. Source Code	7
B. Output Program.....	16
C. Pembahasan	18
D. Tautan Git	21

DAFTAR GAMBAR

Gambar 1. Contoh Penggunaan Debugger	6
Gambar 2.Screenshot Tampilan Aplikasi.....	16
Gambar 3.Screenshot Hasil Logging.....	16
Gambar 4.Screenshot Penggunaan Debugger	17

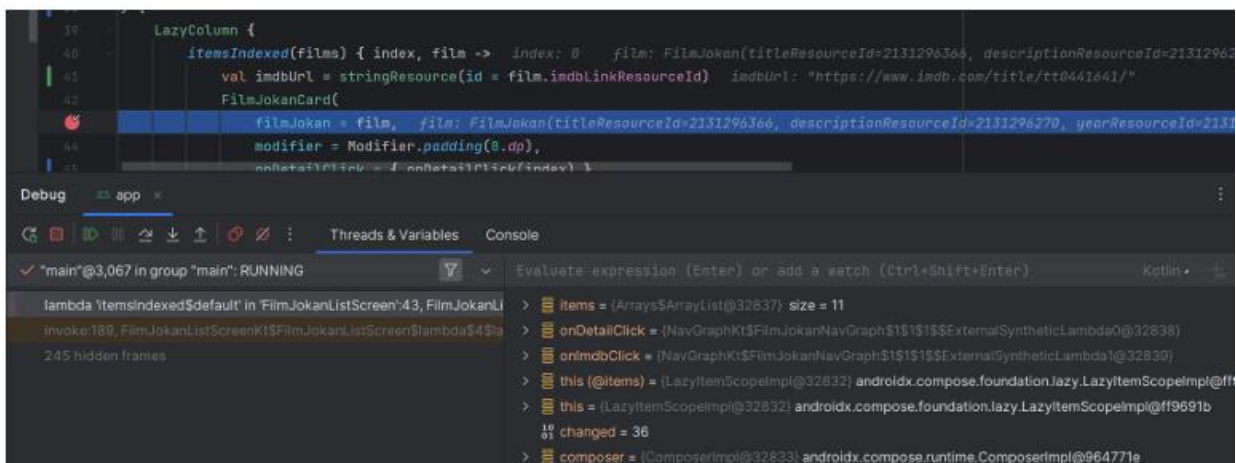
DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1.....	10
Tabel 2. Source Code Jawaban Soal 1.....	12
Tabel 3. Source Code Jawaban Soal 1.....	14
Tabel 4. Source Code Jawaban Soal 1.....	15
Tabel 5. Source Code Jawaban Soal 1.....	15

SOAL

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - b. Gunakan ViewModelFactory dalam pembuatan ViewModel
 - c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
 - d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
 2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya
- Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

1. MainActivity.kt

```
1 package com.example.birthflowerzapp
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.Image
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.res.painterResource
13 import androidx.compose.ui.unit.dp
14 import androidx.navigation.compose.rememberNavController
15 import androidx.navigation.compose.NavHost
16 import androidx.navigation.compose.composable
17 import androidx.navigation.NavController
18 import
19 com.example.birthflowerzapp.ui.theme.BirthFlowerzAppTheme
20 import androidx.compose.foundation.lazy.LazyColumn
21 import androidx.compose.foundation.lazy.items
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.sp
24 import android.content.Intent
25 import android.net.Uri
26 import android.content.Context
27 import androidx.compose.ui.platform.LocalContext
28 import androidx.compose.ui.Alignment
29 import androidx.lifecycle.viewmodel.compose.viewModel
30 import androidx.compose.runtime.collectAsState
31 import com.example.birthflowerzapp.BirthFlowerViewModel
32 import com.example.birthflowerzapp.BirthFlowerViewModelFactory
33
34 class MainActivity : ComponentActivity() {
35     override fun onCreate(savedInstanceState: Bundle?) {
36         super.onCreate(savedInstanceState)
37         setContent {
38             BirthFlowerzAppTheme {
39                 val navController = rememberNavController()
40
41                 NavHost(navController = navController,
42 startDestination = "main_screen") {
43                     composable("main_screen") {
44                         MainScreen(navController =
45 navController)
46                     }
47                     composable("flower_detail/{flowerId}") {
48 backStackEntry ->
49
```

```

50             val flowerId =
51 backStackEntry.arguments?.getString("flowerId")?.toInt() ?: 0
52             FlowerDetailScreen(flower =
53 flowersList[flowerId])
54         }
55     }
56 }
57 }
58 }
59 }
60
61 @Composable
62 fun MainScreen(navController: NavController) {
63     val viewModel: BirthFlowerViewModel = viewModel(factory =
64 BirthFlowerViewModelFactory())
65     val flowers = viewModel.flowers.collectAsState()
66
67     LazyColumn(modifier = Modifier.padding(16.dp)) {
68         items(flowers.value.size) { index ->
69             val flower = flowers.value[index]
70             FlowerItem(
71                 flower = flower,
72                 index = index,
73                 viewModel = viewModel,
74                 navController = navController
75             )
76         }
77     }
78 }
79
80 @Composable
81 fun FlowerItem(
82     flower: Flowers,
83     index: Int,
84     viewModel: BirthFlowerViewModel,
85     navController: NavController
86 ) {
87     val context = LocalContext.current
88
89     Card(
90         shape = RoundedCornerShape(8.dp),
91         modifier = Modifier
92             .fillMaxWidth()
93             .padding(8.dp)
94     ) {
95         Row(
96             modifier = Modifier
97                 .fillMaxWidth()
98                 .padding(16.dp),
99             verticalAlignment = Alignment.CenterVertically
100         ) {
101             Image(

```



```

102         painter = painterResource(id = flower.image),
103         contentDescription = flower.name,
104         modifier = Modifier
105             .size(140.dp)
106     )
107
108     Spacer(modifier = Modifier.width(16.dp))
109
110     Column(
111         modifier = Modifier.weight(1f)
112     ) {
113         Text(
114             text = flower.name,
115             style =
116                 MaterialTheme.typography.headlineMedium.copy(
117                     fontWeight = FontWeight.Bold
118                 )
119         )
120
121         Spacer(modifier = Modifier.height(4.dp))
122
123         Text(
124             text = "Detail:",
125             style =
126                 MaterialTheme.typography.bodyMedium.copy(
127                     fontWeight = FontWeight.Bold
128                 )
129         )
130
131         Spacer(modifier = Modifier.height(4.dp))
132
133         Text(
134             text = flower.symbol,
135             style =
136                 MaterialTheme.typography.bodyMedium
137             )
138
139         Spacer(modifier = Modifier.height(8.dp))
140
141         Row(
142             horizontalArrangement =
143                 Arrangement.spacedBy(8.dp)
144         ) {
145             Button(
146                 onClick = { openLink(flower.wikiLink,
147                     context) },
148                 modifier = Modifier
149                     .width(75.dp)
150             ) {
151                 Text(
152                     text = "Wiki",
153                     fontSize = 12.sp,

```

```

154         maxLines = 1
155     )
156 }
157
158     Button(
159         onClick = {
160             viewModel.selectFlower(index) //
161 Logging klik detail
162
163 navController.navigate("flower_detail/$index")
164         },
165         modifier = Modifier
166             .width(90.dp)
167     ) {
168         Text(
169             text = "Detail",
170             fontWeight = FontWeight.Bold,
171             fontSize = 12.sp,
172             maxLines = 1
173         )
174     }
175 }
176 }
177 }
178 }
179 }
180
181 fun openLink(url: String, context: Context) {
182     val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
183     context.startActivity(intent)
184 }

```

Tabel 1. Source Code Jawaban Soal 1

2. Flowers.kt

```

1 package com.example.birthflowerzapp
2
3 import androidx.annotation.DrawableRes
4
5 data class Flowers(
6     val name: String,
7     val symbol: String,
8     val wikiLink: String,
9     @DrawableRes val image: Int
10 )
11
12
13 val flowersList = listOf(
14     Flowers("January - Carnation",
15

```

```

16         "Symbolizes admiration and love. Represents deep
17 affection and distinction.",
18
19     "https://en.wikipedia.org/wiki/Dianthus_caryophyllus",
20     R.drawable.carnation),
21
22     Flowers("February - Iris",
23         "Represents wisdom and hope. Known for elegance
24 and royal symbolism.",
25         "https://en.wikipedia.org/wiki/Iris_(plant)",
26         R.drawable.iris),
27
28     Flowers("March - Daffodil",
29         "Symbol of new beginnings and hope. Blooms mark
30 the start of spring.",
31         "https://en.wikipedia.org/wiki/Narcissus_(plant)",
32         R.drawable.daffodil),
33
34     Flowers("April - Sweet Pea",
35         "Conveys bliss and goodbyes. Fragrant flower
36 representing gentle charm.",
37         "https://en.wikipedia.org/wiki/Lathyrus_odoratus",
38         R.drawable.sweetpea),
39
40     Flowers("May - Lily of the Valley",
41         "Represents purity and joy. White blooms
42 symbolize fresh starts.",
43         "https://en.wikipedia.org/wiki/Lily_of_the_valley",
44         R.drawable.lilyofthevalley),
45
46     Flowers("June - Rose",
47         "Classic symbol of love and beauty. Each color
48 has its own meaning.",
49         "https://en.wikipedia.org/wiki/Rose",
50         R.drawable.rose),
51
52     Flowers("July - Larkspur",
53         "Stands for lightness and joy. Bright spikes
54 represent strong bonds.",
55         "https://en.wikipedia.org/wiki/Delphinium",
56         R.drawable.larkspur),
57
58     Flowers("August - Poppy",
59
60
61
62
63
64
65
66
67

```

68	"Symbolizes peace and remembrance. Red petals
69	evoke reflection.",
70	"https://en.wikipedia.org/wiki/Poppy",
71	R.drawable.poppy),
72	
73	Flowers("September - Aster",
74	"Represents love and faith. Star-shaped petals
75	show lasting beauty.",
76	"https://en.wikipedia.org/wiki/Aster",
77	R.drawable.aster),
78	
79	
80	Flowers("October - Marigold",
81	"Stands for warmth and creativity. Golden tones
82	bring optimism.",
83	"https://en.wikipedia.org/wiki/Tagetes",
84	R.drawable.marigold),
85	
86	
87	Flowers("November - Peony",
88	"Symbolizes romance and honor. Lush blooms
89	express elegance.",
90	"https://en.wikipedia.org/wiki/Peony",
91	R.drawable.peony),
92	
93	Flowers("December - Narcissus",
94	"Represents self-love and rebirth. A flower of
95	renewal and growth.",
96	
97	"https://en.wikipedia.org/wiki/Narcissus_(plant)",
98	R.drawable.narcissus)
	)

Tabel 2. Source Code Jawaban Soal 1

3. FlowerDetailActivity.kt

1	package com.example.birthflowerzapp
2	
3	import android.os.Bundle
4	import android.util.Log
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.compose.foundation.Image
8	import androidx.compose.foundation.layout.*
9	import androidx.compose.foundation.lazy.LazyColumn
10	import androidx.compose.foundation.lazy.items
11	import androidx.compose.material3.*
12	import androidx.compose.runtime.Composable
13	import androidx.compose.ui.Modifier
14	

```

15 import androidx.compose.ui.res.painterResource
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.text.font.FontWeight
18 import androidx.compose.ui.unit.sp
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.text.style.TextAlign
21
22
23
24 class FlowerDetailActivity : ComponentActivity() {
25     override fun onCreate(savedInstanceState: Bundle?) {
26         super.onCreate(savedInstanceState)
27         setContent {
28             val flowerId = intent.getIntExtra("flowerId",
29 0)
30             val flower = flowersList[flowerId]
31             FlowerDetailScreen(flower = flower)
32         }
33     }
34 }
35
36
37 @Composable
38 fun FlowerDetailScreen(flower: Flowers) {
39     Log.d("FlowerDetailScreen", "Menampilkan detail
40 bunga: ${flower.name}")
41
42     LazyColumn(
43         modifier = Modifier
44             .padding(16.dp)
45             .fillMaxSize(),
46         verticalArrangement = Arrangement.Top,
47         horizontalAlignment =
48 Alignment.CenterHorizontally
49     ) {
50         items(listOf(flower)) { currentFlower ->
51             Image(
52                 painter = painterResource(id =
53 currentFlower.image),
54                 contentDescription = currentFlower.name,
55                 modifier = Modifier.size(300.dp)
56             )
57             Spacer(modifier = Modifier.height(16.dp))
58
59             Text(
60                 text = currentFlower.name,
61                 style =
62 MaterialTheme.typography.headlineLarge.copy(
63                     fontWeight = FontWeight.Bold,
64

```

67	fontSize = 28.sp
68	)
69	)
70	Spacer(modifier = Modifier.height(8.dp))
71	
72	Text(
73	text = "Detail:",
74	style =
75	MaterialTheme.typography.bodyLarge.copy(
76	fontWeight = FontWeight.Bold
77	),
78	modifier = Modifier.fillMaxWidth(),
79	textAlign = TextAlign.Start
80	)
81	
82	
83	Spacer(modifier = Modifier.height(4.dp))
84	
85	Text(
86	text = flower.symbol,
87	style =
88	MaterialTheme.typography.bodyLarge
89	)
90	
91	}
92	}
93	}
94	

Tabel 3. Source Code Jawaban Soal 1

4. BirthFlowerViewModel.kt

1	package com.example.birthflowerzapp
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import android.util.Log
7	
8	
9	class BirthFlowerViewModel : ViewModel() {
10	
11	private val _flowers = MutableStateFlow(flowersList)
12	val flowers: StateFlow<List<Flowers>> get() =
13	_flowers
14	
15	private val _selectedFlowerIndex =
16	MutableStateFlow<Int?>(null)
17	val selectedFlowerIndex: StateFlow<Int?> get() =
18	_selectedFlowerIndex
19	
20	

21	fun selectFlower(index: Int) {
22	_selectedFlowerIndex.value = index
23	Log.d("BirthFlowerViewModel", "Tombol Detail
24	ditekan untuk bunga: \${flowersList[index].name}")
25	}
26	
27	init {
28	Log.d("BirthFlowerViewModel", "Data bunga
29	ditambahkan: \${flowersList.size} item")
30	}
31	}
32	}
33	
34	

Tabel 4. Source Code Jawaban Soal 1

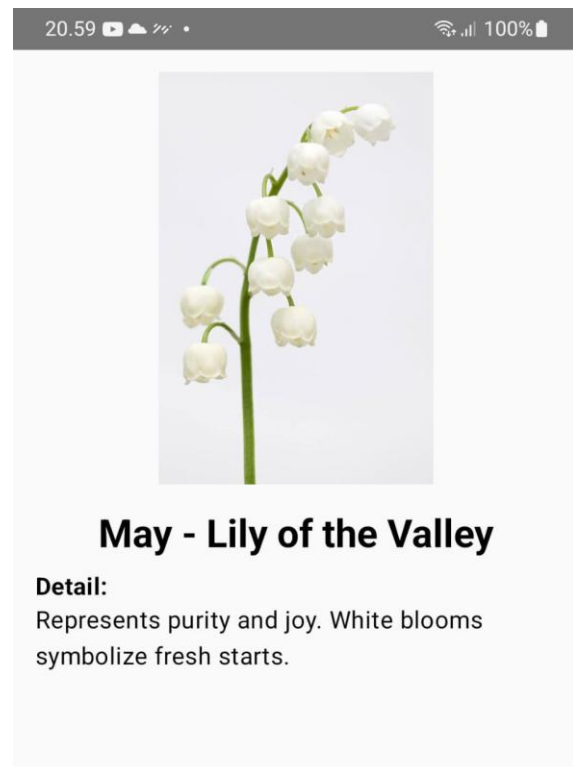
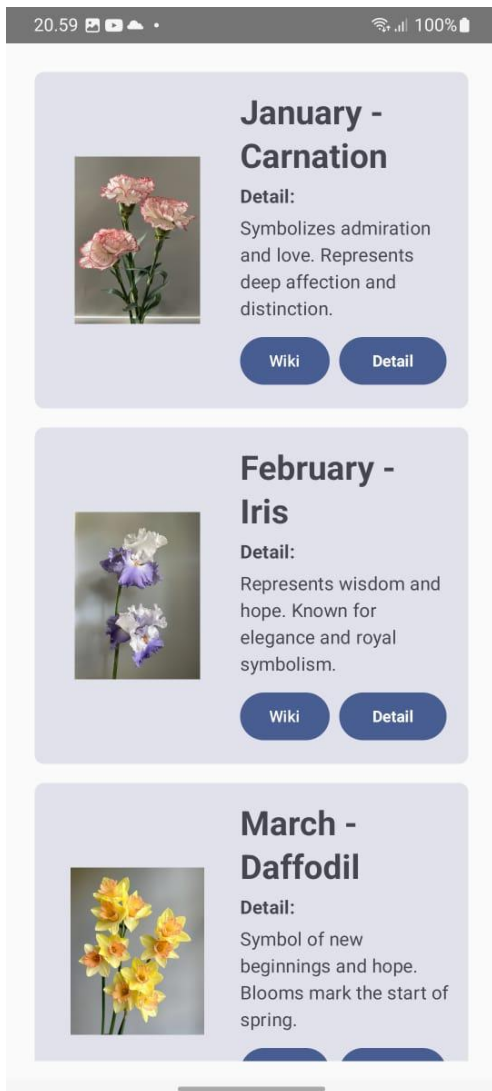
5. BirthFlowerViewModelFactory.kt

1	package com.example.birthflowerzapp
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class BirthFlowerViewModelFactory : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass: Class<T>):
8	T {
9	if
10	(modelClass.isAssignableFrom(BirthFlowerViewModel::class.java))
11	{
12	return BirthFlowerViewModel() as T
13	}
14	throw IllegalArgumentException("Unknown ViewModel
15	class")
16	}
17	}
18	}
19	
20	

Tabel 5. Source Code Jawaban Soal 1

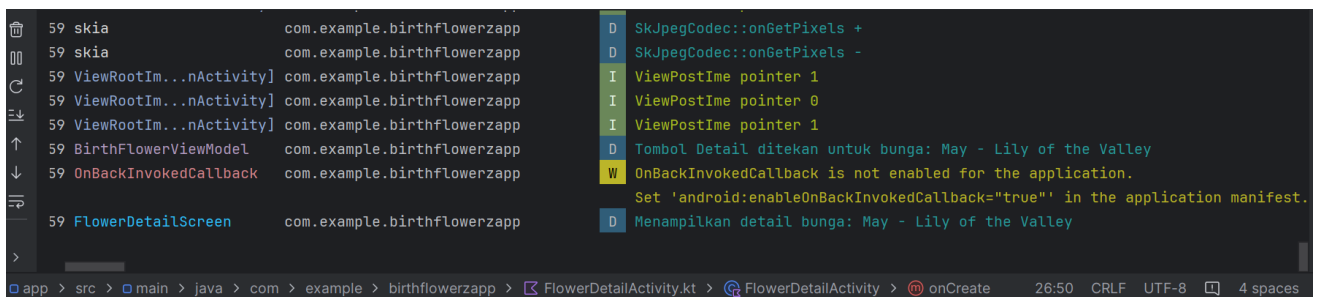
B. Output Program

Tampilan Aplikasi:



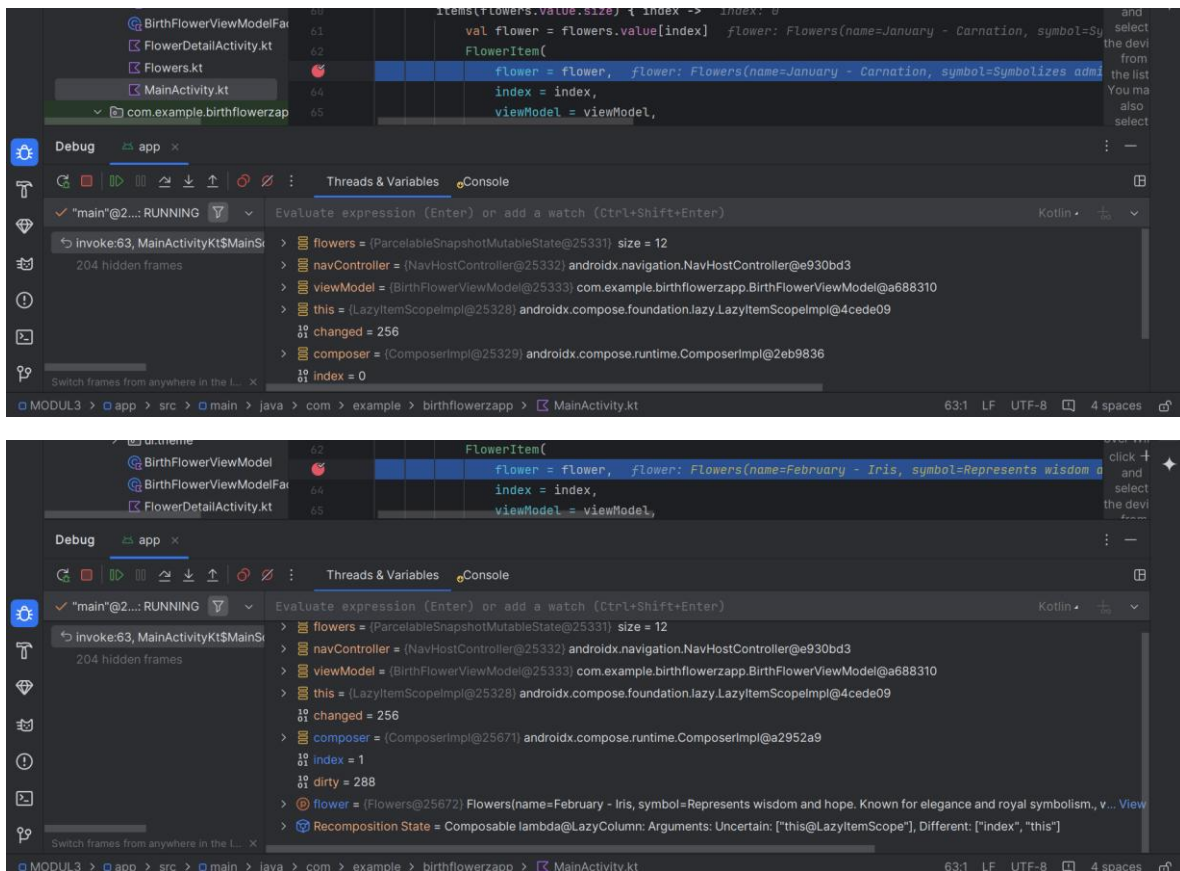
Gambar 2.Screenshot Tampilan Aplikasi

Logcat:



Gambar 3.Screenshot Hasil Logging

Debugger:



Gambar 4. Screenshot Penggunaan Debugger

1. Debugger adalah alat bantu dalam Android Studio yang digunakan untuk:

- Menjalankan aplikasi baris per baris
- Melihat isi variabel secara langsung saat aplikasi berjalan
- Membantu menemukan bug atau kesalahan logika dalam kode

2. Cara Menggunakan Debugger:

- Tambahkan breakpoint (titik merah) di baris kode penting, misalnya saat fungsi dijalankan.
- Jalankan aplikasi dengan menekan tombol Debug.
- Saat eksekusi mencapai breakpoint, aplikasi akan berhenti sementara (pause).
- Gunakan fitur Step untuk menelusuri alur program.

3. Fitur Step:

- Step Into: Masuk ke dalam fungsi yang dipanggil untuk melihat isi detailnya.
- Step Over: Melewati fungsi tanpa masuk ke dalamnya, lanjut ke baris berikutnya.
- Step Out: Keluar dari fungsi sekarang dan kembali ke tempat fungsi itu dipanggil.

Pada gambar screenshot debug saya, breakpoint aktif di MainActivity.kt saat FlowerItem() dijalankan:

- Variabel seperti index, flower, dan viewModel bisa dilihat nilainya langsung.
- Fitur Step Into, Step Over, dan Step Out bisa digunakan untuk melanjutkan eksekusi program sambil menelusuri logika kode.

C. Pembahasan

1. MainActivity.kt:

Ini adalah file utama yang menjalankan aplikasi dan menampilkan antarmuka pengguna. Di dalamnya terdapat `onCreate()` yang memanggil `setContent` untuk menampilkan tampilan berbasis Jetpack Compose. Aplikasi ini menggunakan navigasi Compose untuk berpindah antar halaman. Di dalam `NavHost`, terdapat dua route: `main_screen` dan `flower_detail/{flowerId}`. Halaman `main_screen` menampilkan daftar bunga, sedangkan halaman `flower_detail` menampilkan detail bunga berdasarkan ID yang dikirim. Fungsi `MainScreen` mengambil data dari `ViewModel` dengan cara memanggil `viewModel(factory = BirthFlowerViewModelFactory())`. Data bunga yang tersimpan dalam `ViewModel` disimpan menggunakan `StateFlow`, lalu ditampilkan ke layar dengan `collectAsState()`. Data tidak lagi disimpan langsung di `Activity`, melainkan dikelola secara terpisah oleh `ViewModel` agar lebih aman dan tetap tersimpan saat layar diputar atau navigasi berpindah. Setiap item bunga ditampilkan dengan fungsi `FlowerItem`, yang berisi gambar, nama bunga, deskripsi singkat, dan dua tombol.

Tombol "Wiki" membuka halaman Wikipedia menggunakan `Intent`, sedangkan tombol "Detail" memanggil fungsi `selectFlower(index)` dari `ViewModel` dan berpindah ke halaman detail. Di sinilah praktik logging dan debugging mulai diterapkan, karena saat tombol ini ditekan, log dicatat di `Logcat`, dan kita bisa memasang breakpoint di bagian ini saat debugging.

2. `Flowers.kt`

Kode ini berisi data class `Flowers` yang mendefinisikan struktur data bunga, yaitu `name`, `symbol`, `wikiLink`, dan `image`. Di bawahnya terdapat `flowersList`, yaitu daftar lengkap bunga dari bulan Januari hingga Desember, masing-masing dengan informasi nama, arti, link Wikipedia, dan gambar yang tersimpan di `drawable`. Data ini sebelumnya langsung digunakan di `MainActivity`, tetapi setelah penerapan Modul 4, list ini dipindahkan ke dalam `ViewModel` agar lebih aman dan sesuai prinsip pemisahan logika dan tampilan (*separation of concern*).

3. `FlowerDetailActivity.kt`

File ini digunakan untuk menampilkan detail bunga ketika pengguna mengklik tombol "Detail". Di dalam fungsi `onCreate`, aplikasi mengambil `flowerId` dari `intent`, lalu menggunakan ID tersebut untuk mengambil objek bunga dari daftar. Bagian utama tampilannya ditulis dalam fungsi `FlowerDetailScreen`, yang menampilkan gambar bunga, nama bunga, dan penjelasannya. Di awal fungsi ini juga terdapat perintah `Log.d(...)` untuk mencatat bahwa pengguna sedang melihat detail bunga tertentu. Log ini penting untuk keperluan debugging, karena kita bisa memantau interaksi pengguna dan menelusuri masalah jika ada tampilan atau data yang tidak sesuai.

4. `BirthFlowerViewModel.kt`

File ini berisi list bunga (`flowersList`) disimpan dalam sebuah `StateFlow` bernama `_flowers`, sehingga data bisa di-observe dan diperbarui jika perlu. Selain itu, `ViewModel` ini juga menyimpan `selectedFlowerIndex` — index bunga yang terakhir dipilih oleh pengguna. Fungsi `selectFlower(index)` akan menyimpan index bunga ke dalam

StateFlow dan mencatat log di Logcat. Fungsi ini dipanggil setiap kali pengguna menekan tombol “Detail”. Karena ViewModel tidak bergantung pada UI secara langsung, maka jika layar diputar atau UI di-recompose, data bunga tetap tersimpan dan tidak hilang. Ini adalah keunggulan utama penggunaan ViewModel.

5. BirthFlowerViewModelFactory.kt

File ini untuk memberikan instance `BirthFlowerViewModel` ke dalam Compose UI. Ini juga membantu memisahkan pembuatan ViewModel dari UI, yang membuat kode lebih bersih dan modular.

2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

JAWAB: Application class adalah kelas yang dijalankan pertama kali saat aplikasi Android dibuka, sebelum Activity, Service, atau komponen lainnya.

Fungsi utamanya:

- Menyediakan tempat untuk inisialisasi global, seperti database, library, atau konfigurasi awal.
- Digunakan untuk menyimpan data global yang bisa diakses dari seluruh bagian aplikasi.
- Cocok untuk mengatur logika yang bersifat umum, misalnya logging atau pengaturan bahasa aplikasi.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/rgnsilvas/Pemrograman-Mobile/tree/master/MODUL4>